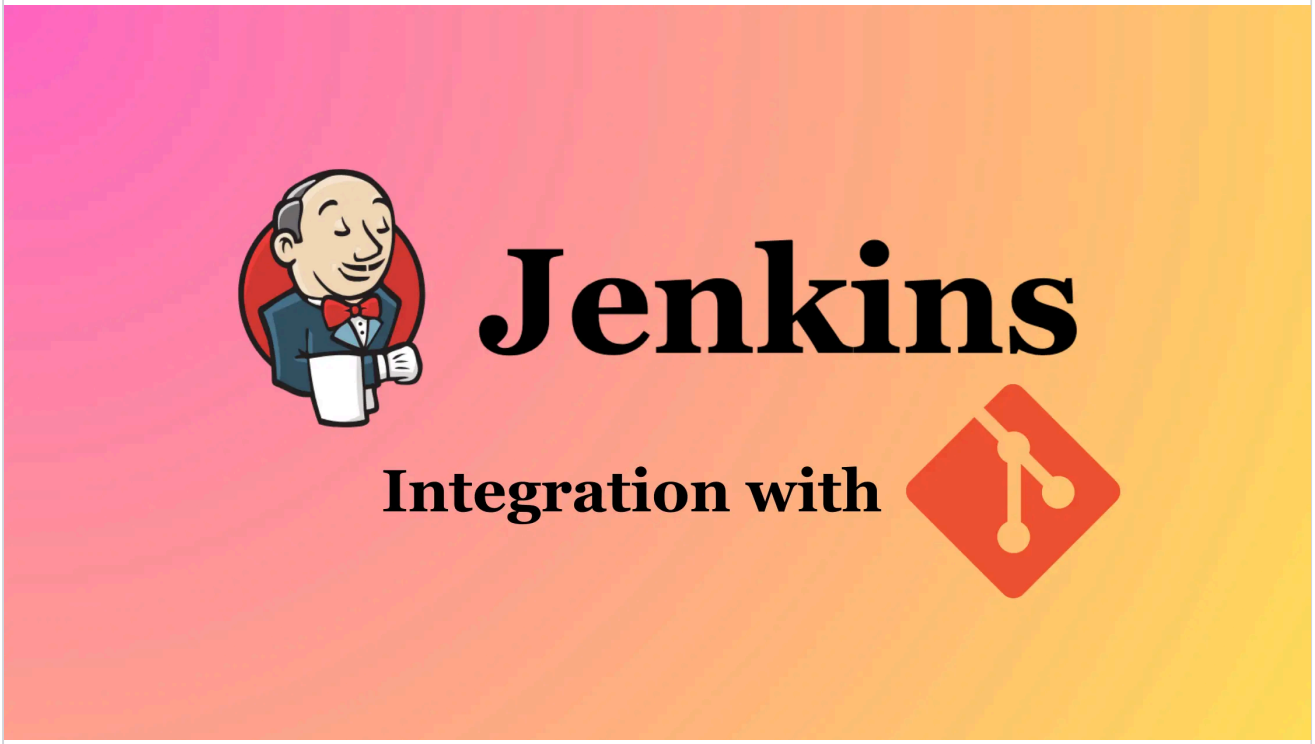
Integrating Git with Jenkins: A Complete Guide

Updated December 21, 2025 • 7 min read



Shaik Mustafa

Introduction

Continuous Integration (CI) is a core practice of modern DevOps. Integrating version control systems like Git with Jenkins empowers teams to automate code **builds, tests & deployments** seamlessly. In this article, we'll explore how to integrate Git with Jenkins and make the things automation.

Contents

Why Integrate Git with Jenkins?

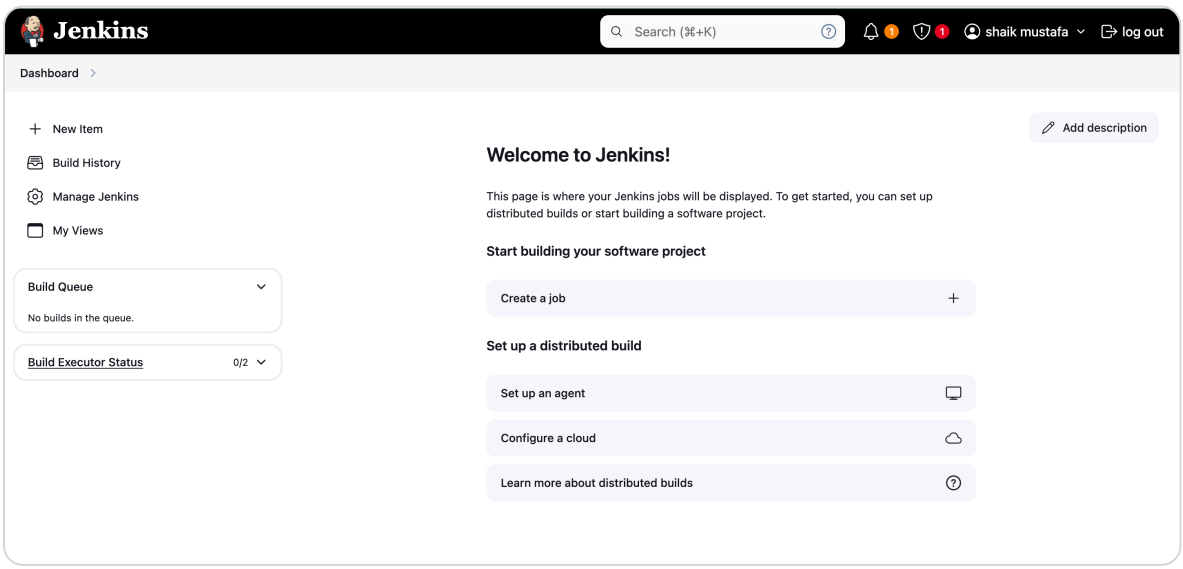
- **Automated Builds:** Automatically trigger builds whenever code changes are pushed to the repository.
- **Seamless Collaboration**: Boost team productivity by catching and resolving issues early in the development pipeline, ensuring smoother workflows and fewer delays.
- **Effortless Scalability**: Streamline processes for large-scale projects, making it easier to manage distributed teams and complex workflows efficiently.

Prerequisites

- **Jenkins installed** and configured on your server.
- **A Git repository** (GitHub or BitBucket).
- Necessary **Jenkins plugins** (e.g., Git Plugin).

Step-by-Step Guide

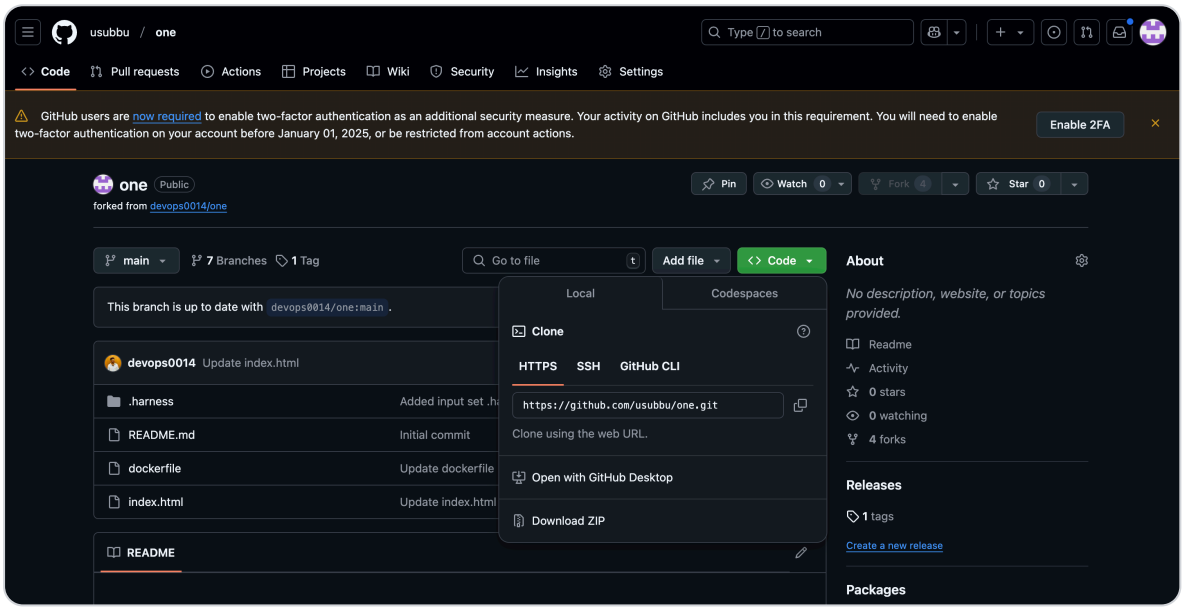
STEP-1: Install Jenkins and setup dashboard



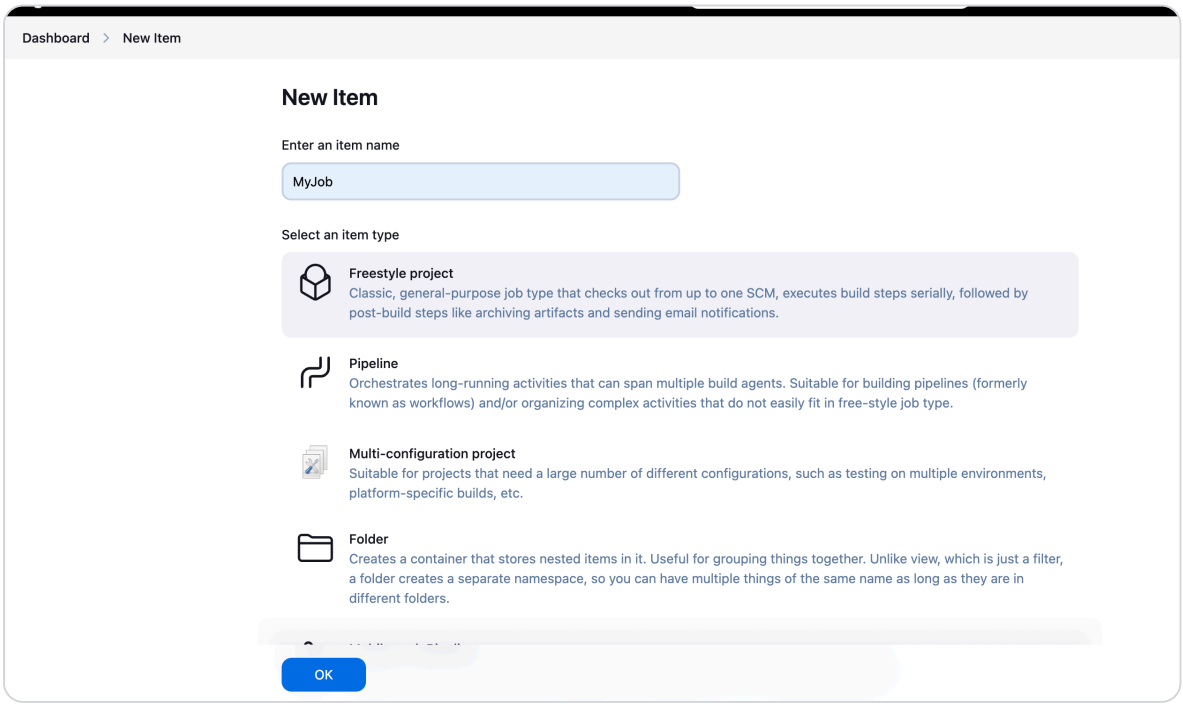
STEP-2: Install git on Jenkins server

```
yum install git -y
```

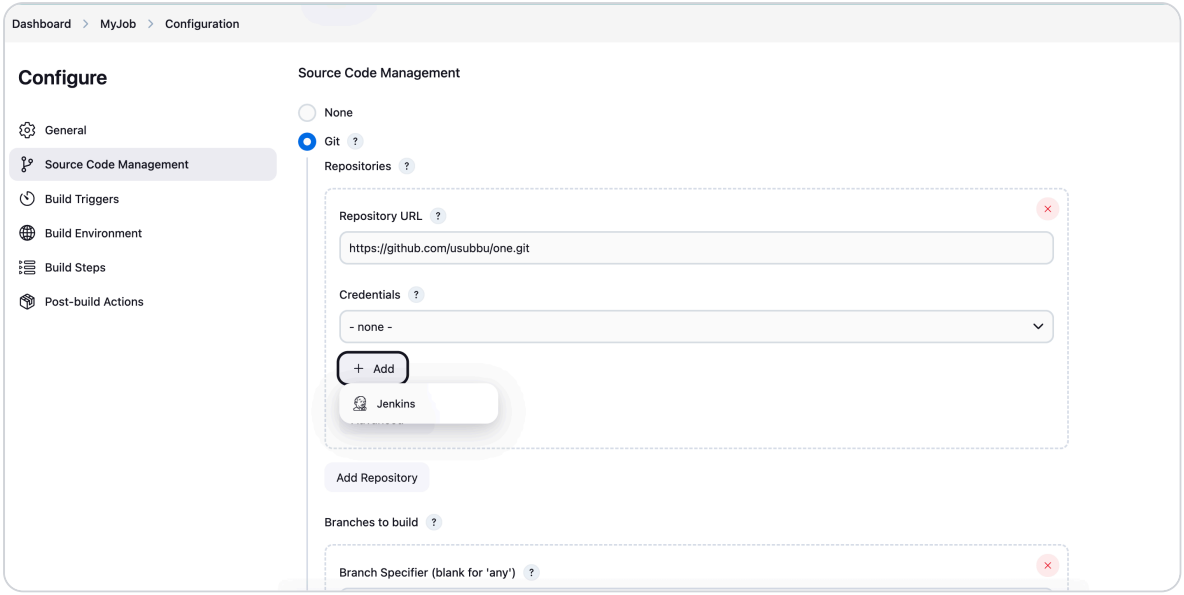
STEP-3: Open your github repo and copy https link (https://github.com/usubbu/one.git)



STEP-4: Go to jenkins dashboard, **create a job**



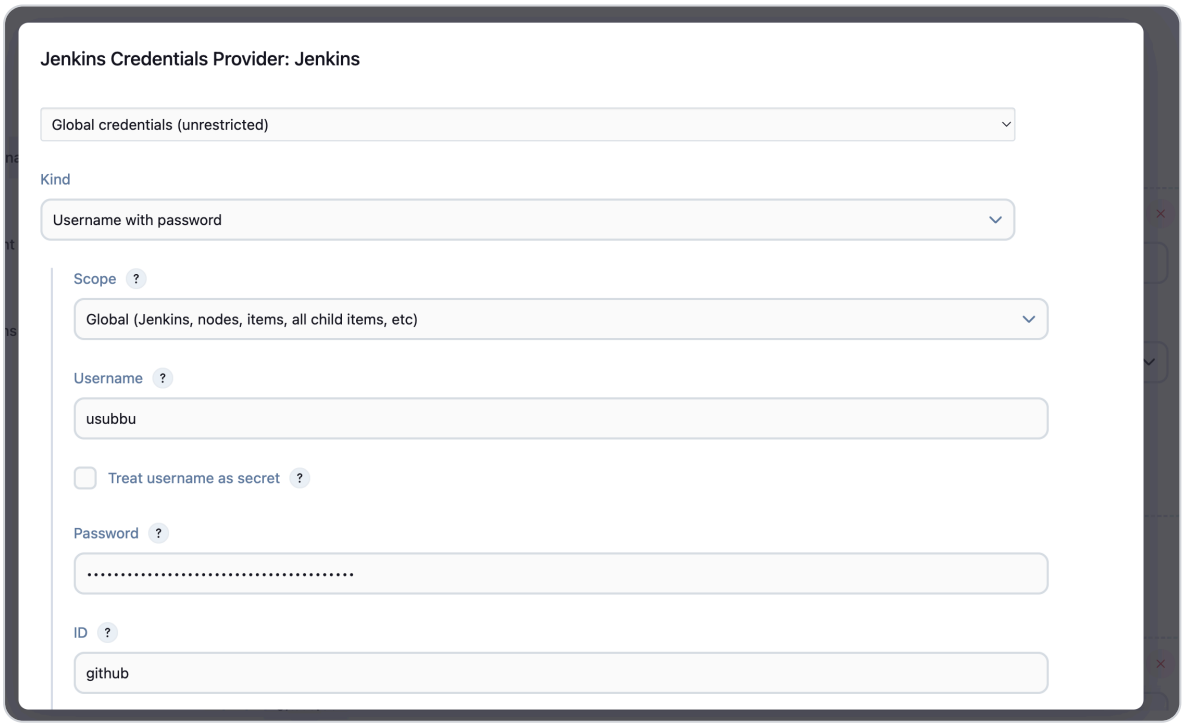
Now go to **source code management** and paste your **repo-url** and add the **credentials**



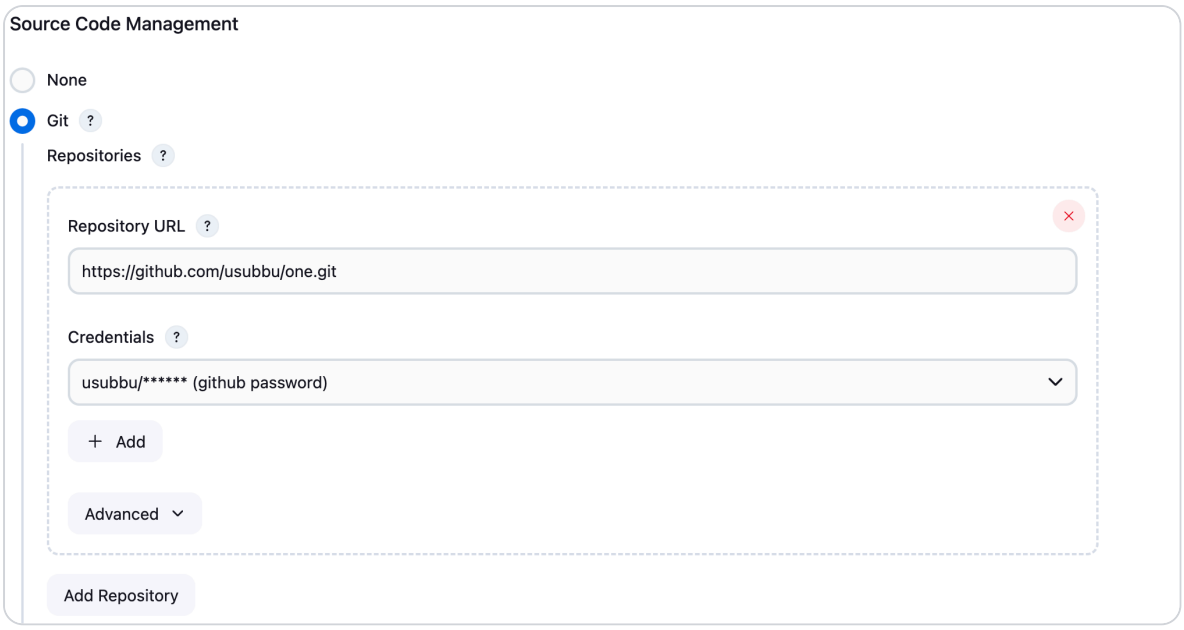
Username : <your github user-name>

Password: <Enter Github Token>

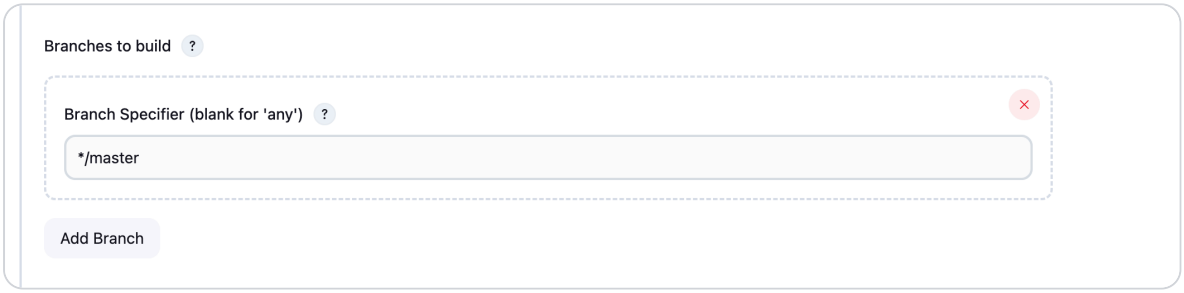
ID: github



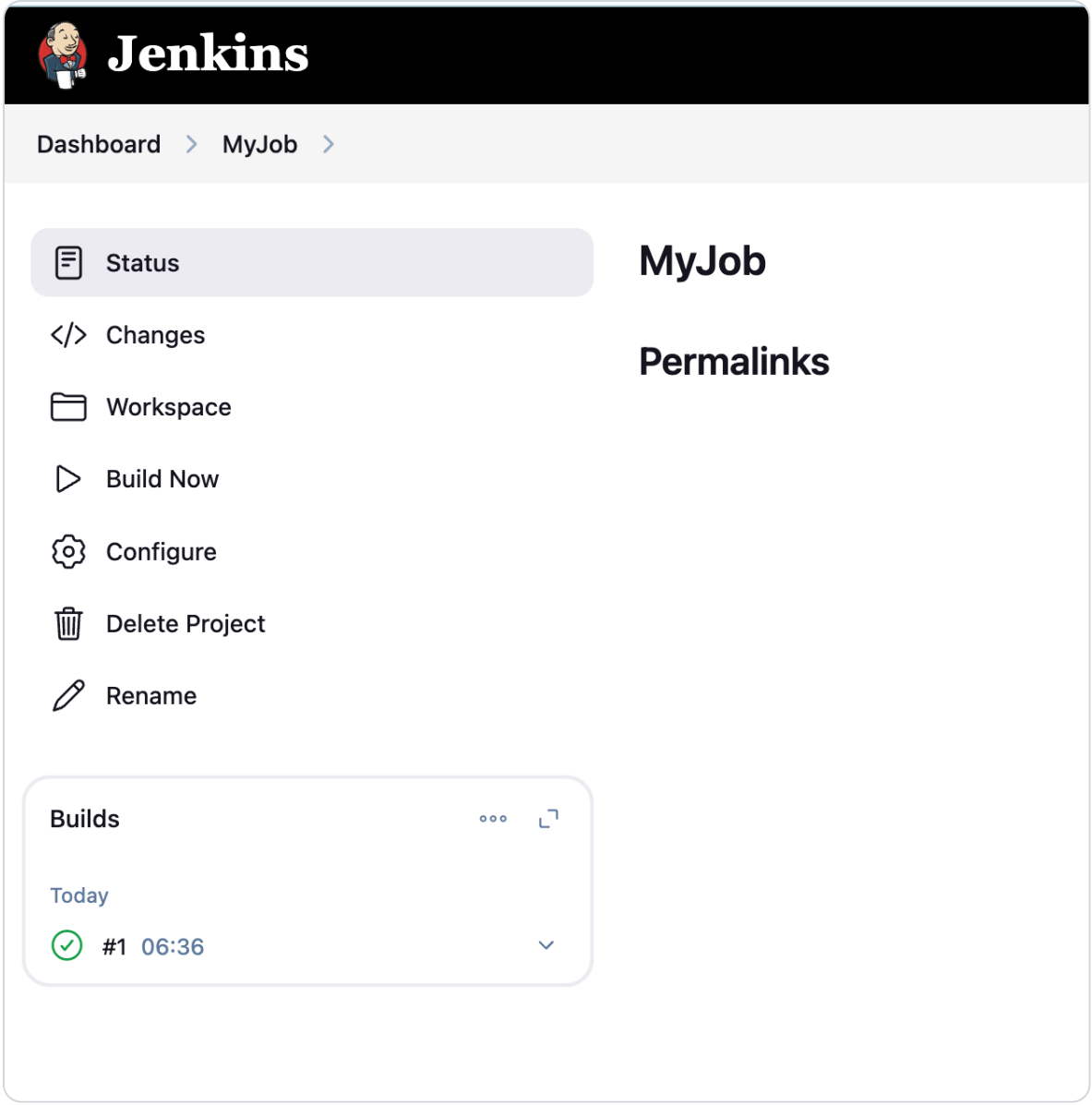
and click on **add**



Now enter the branch name that you want to build. Bydefault it will be **master**, if you wish to build other branch make sure to change the branch name that you want to build.



Now **save** the job and build it.



Hurray! Now we will get all the files from GitHub to CI Server. So developer make any changes in GitHub e have to build here in Jenkins.

Now lets make it automate - Whenever developer make some changes in GitHub, Jenkins has to trigger the build automatically. To implement it we can use the below concepts

1. Webhooks
2. Poll SCM
3. Build Periodically

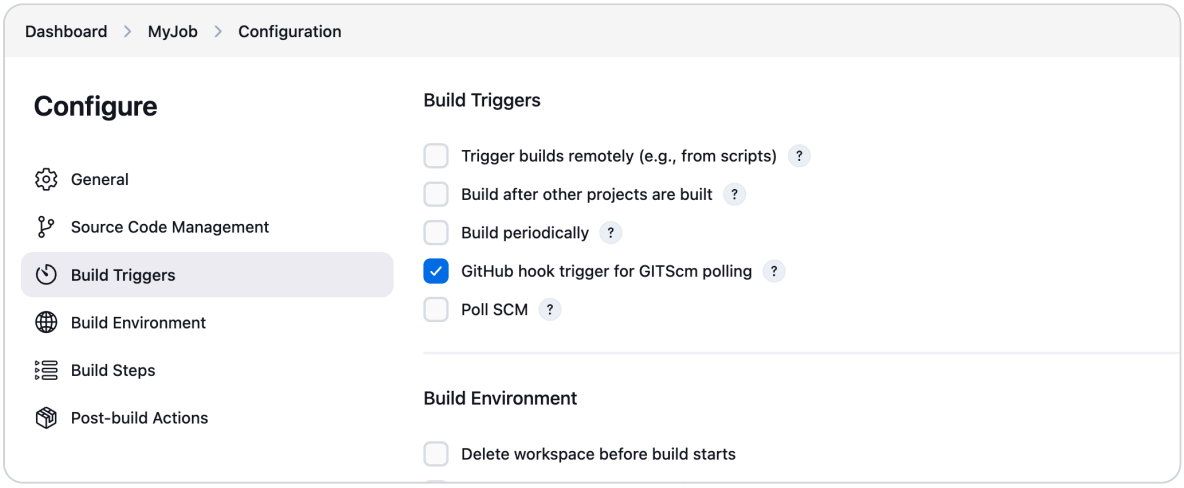
What is a Webhook?

A **webhook** is a way for one application to send real-time data to another application whenever a specific event occurs. In the context of Git and Jenkins, webhooks enable Jenkins to receive notifications from a Git repository (e.g., GitHub, GitLab) whenever a commit is pushed or a pull request is created.

How to Integrate Webhooks with Jenkins

Step 1: Configure Jenkins Job

1. Go to the Jenkins dashboard and open the job configuration page.
2. Scroll to the **Build Triggers** section.
3. Select **GitHub hook trigger for GITScm polling** (if using GitHub).



Now save the job

Step 2: Set Up Webhooks in the Git Repository

The process varies slightly depending on the Git platform (GitHub, GitLab, Bitbucket). Below is the guide for GitHub:

For GitHub:

1. Go to your GitHub repository.

2. Navigate to **Settings > Webhooks**.

3. Click **Add webhook**.

4. Fill out the form:

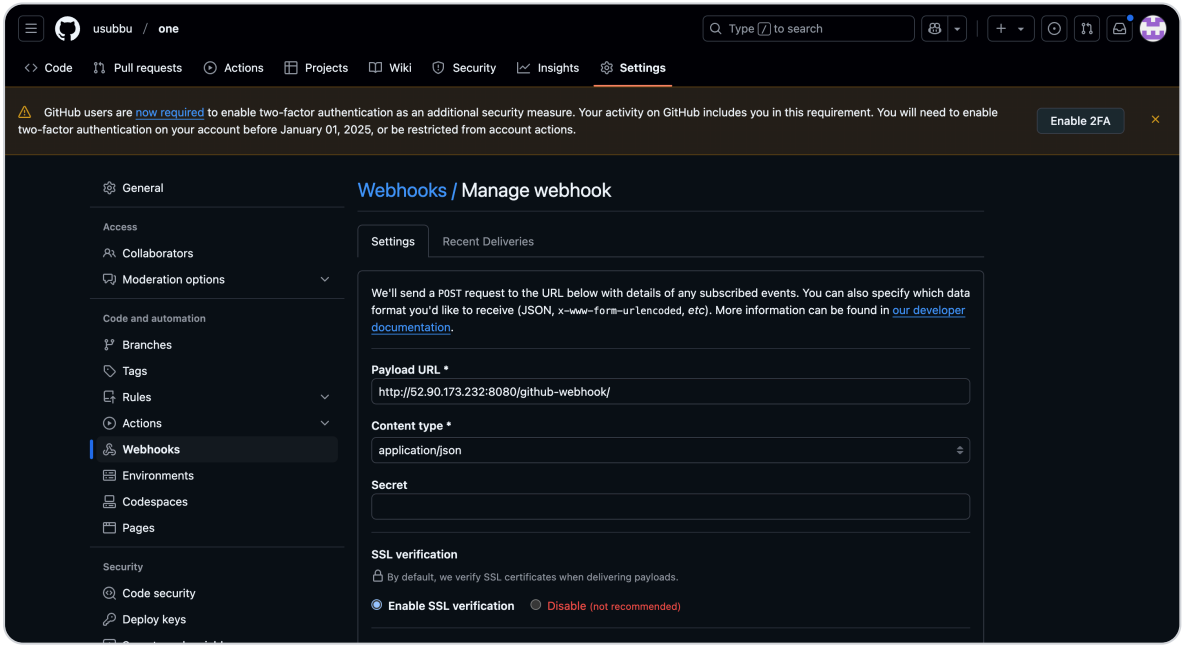
   - **Payload URL:** Enter the Jenkins webhook URL. It typically looks like this:

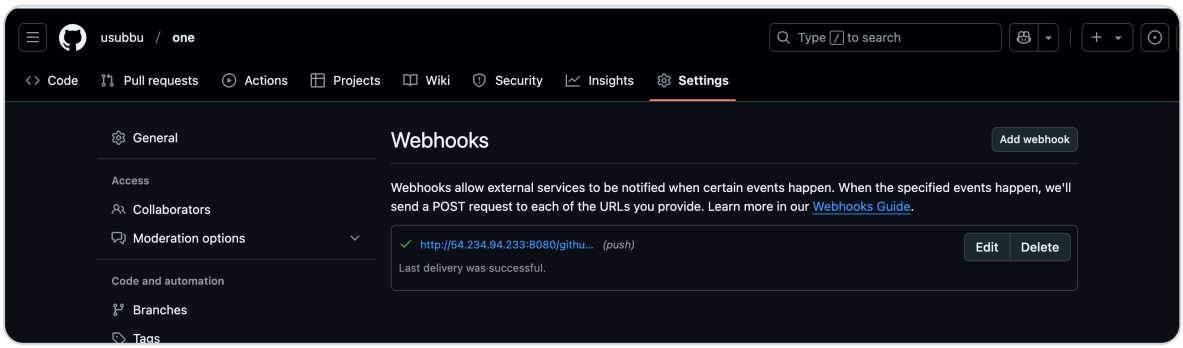
     ```
     http://<JENKINS_URL>:<PORT>/github-webhook/
     ```

     Example: **`http://192.168.1.100:8080/github-webhook/`**

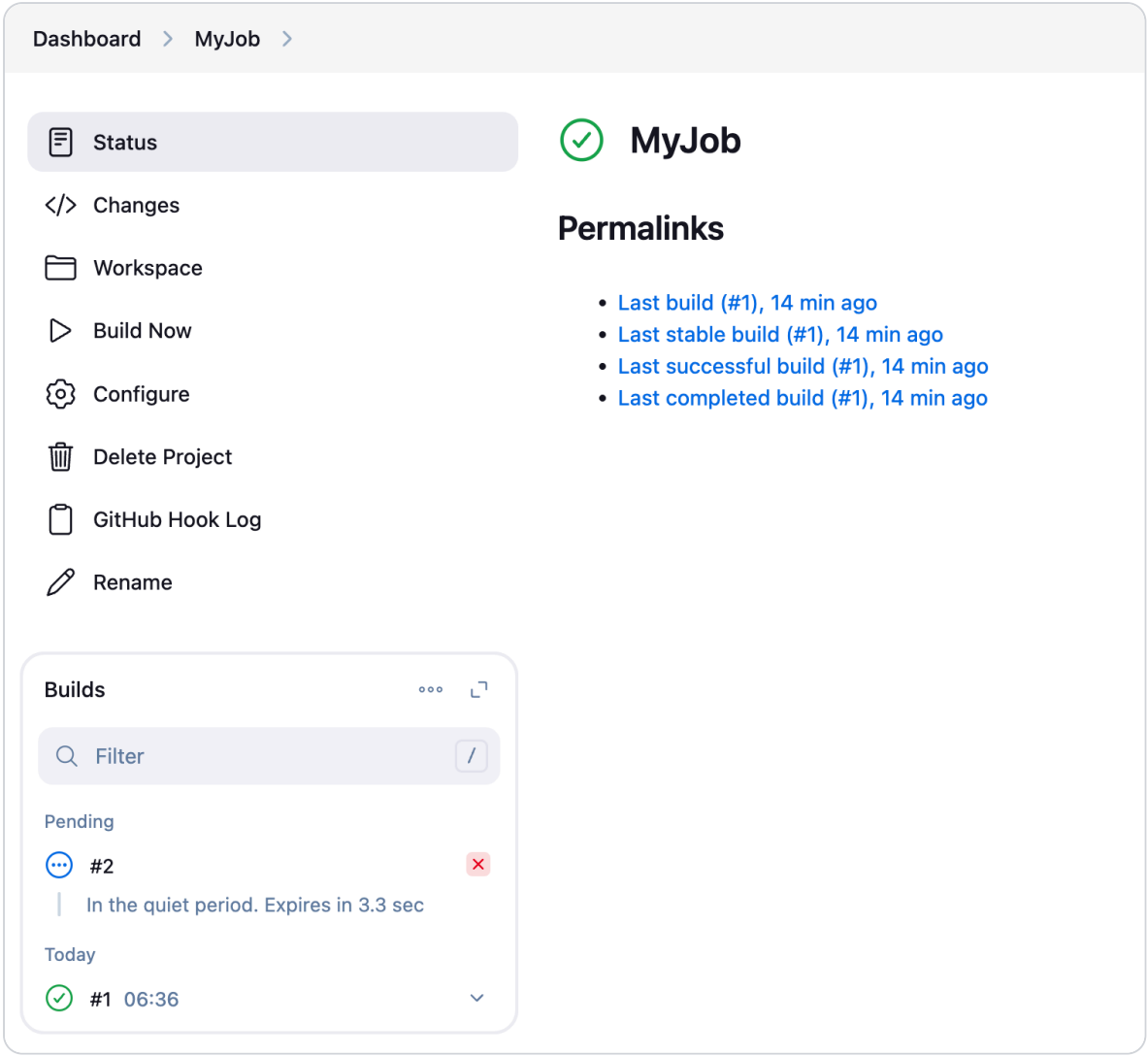
   - **Content Type:** Select **`application/json`** .

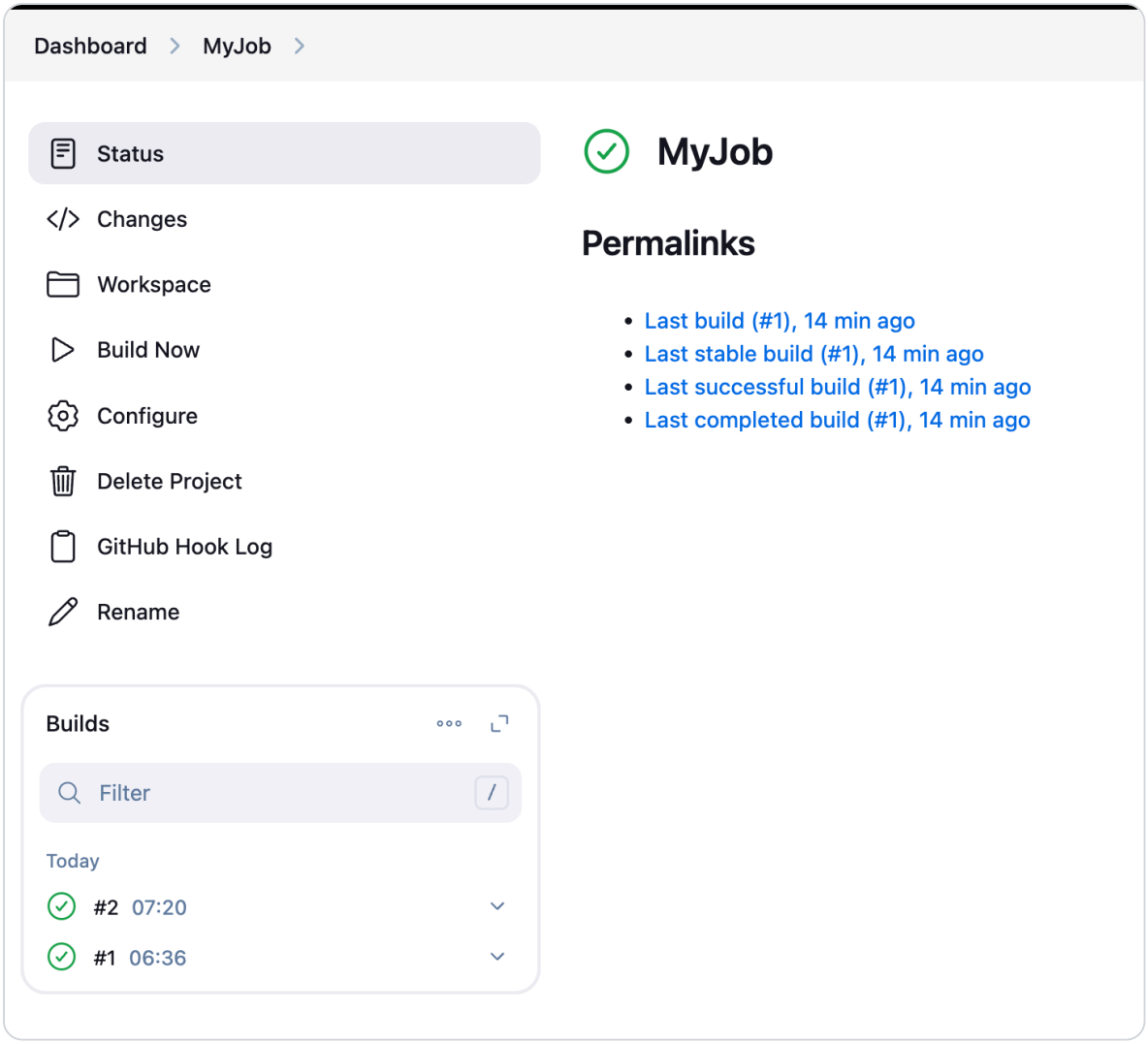
   - Click **Add webhook**.





Now lets make some changes on GitHub, jenkins will automatically trigger the job.





Benefits of Using Webhooks with Jenkins

- **Instant Build Triggers:** Eliminate delays caused by SCM polling.
- **Resource Efficiency:** Save CPU and bandwidth by avoiding periodic polling.
- **Real-Time Feedback:** Developers get faster feedback on code changes.

What is Poll SCM?

Poll SCM is a Jenkins feature used to periodically check a Source Code Management (SCM) repository (like Git) for changes. If any changes are

detected, Jenkins triggers a build. Unlike webhooks (which push changes to Jenkins in real time), Poll SCM uses a schedule-based mechanism to pull updates from the repository.

How Does Poll SCM Work?

1. Jenkins queries the repository at regular intervals (as per the configured schedule).
2. It checks for new commits, branch updates, or tags.
3. If a change is found, Jenkins triggers the associated job or pipeline.

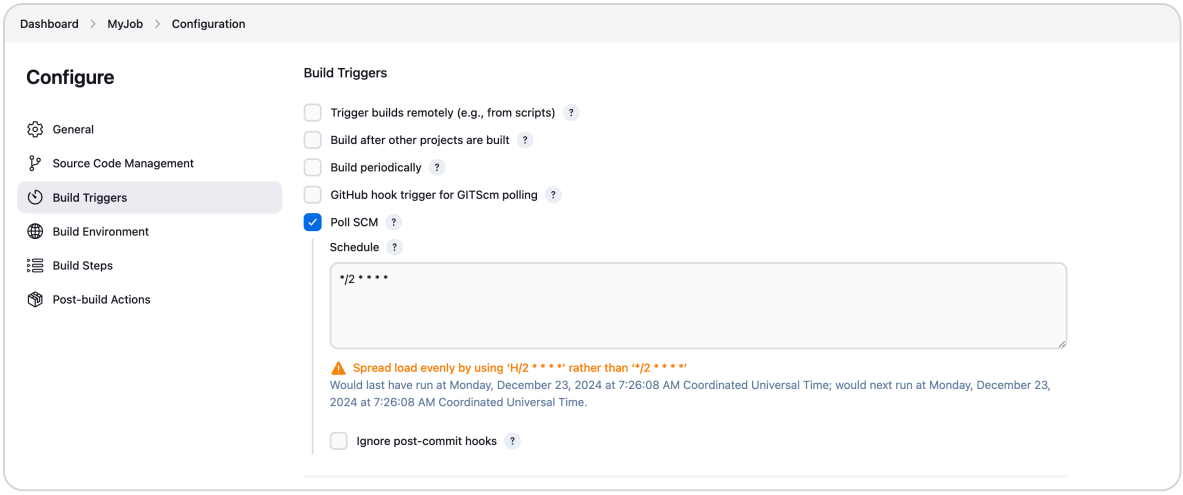
Advantages of Poll SCM:

- **Simple Setup**: No external configurations like webhooks are needed.
- **Supports All SCMs**: Works with any source code management system supported by Jenkins.

How to Configure Poll SCM in Jenkins

Step 1: Configure Jenkins Job

1. Go to the Jenkins dashboard and open the job configuration page.
2. Scroll to the **Build Triggers** section.
3. Select the checkbox for **Poll SCM**.
4. In the **Schedule** field, define the polling schedule using Cron syntax.



I have scheduled the job for every 2nd minute using Cron syntax, Now Jenkins automatically check for every 2 minutes weather any changes are available on github or not. If Jenkins found new commits then it will trigger the jobs.

What is Cron Syntax?

Cron syntax is a format used to define schedules for recurring tasks. In Jenkins, it is used to schedule tasks like polling a source code repository or triggering builds at specific intervals.

A Cron expression consists of **five fields** that specify the time and frequency of execution:

Field	Allowed Values	Description
MINUTE	`0-59`	Minute of the hour
HOUR	`0-23`	Hour of the day
DOM	`1-31`	Day of the month
MONTH	`1-12` (or names)	Month (e.g., `1 = January` , `2 = February`)
DOW	`0-7` (or names)	Day of the week (`0/7 = Sunday`)

- Use `*` to represent "every value."
- Use `H` for hashed values (randomized to distribute workload).

Examples of Cron Syntax in Jenkins

1. **Every 5 minutes**:

```
H/5 * * * *
```

2. **Every day at midnight**:

```
0 0 * * *
```

3. **Every Monday at 9:00 AM**:

```
0 9 * * 1
```

4. **Every 15 minutes during office hours (9 AM to 5 PM)**:

```
H/15 9-17 * * *
```

5. **On the 1st and 15th of every month at 6:00 PM**:

```
0 18 1,15 * *
```

6. **At 3:30 PM on weekdays**:

```
30 15 * * 1-5
```

What is Build Periodically in Jenkins?

Build Periodically is a feature in Jenkins that allows you to schedule builds at regular intervals without relying on external events like code changes in the repository or webhook triggers. It uses a **cron-like syntax** to define the schedule, enabling automated and recurring builds at specific times.

Key Use Cases of Build Periodically:

1. **Scheduled Jobs:** Run nightly builds or periodic tasks, such as regression tests, backups, or report generation.
2. **Environment Maintenance:** Schedule regular updates for infrastructure or deployment pipelines.
3. **Testing Pipelines:** Continuously verify your builds and deployments even when there are no code changes.

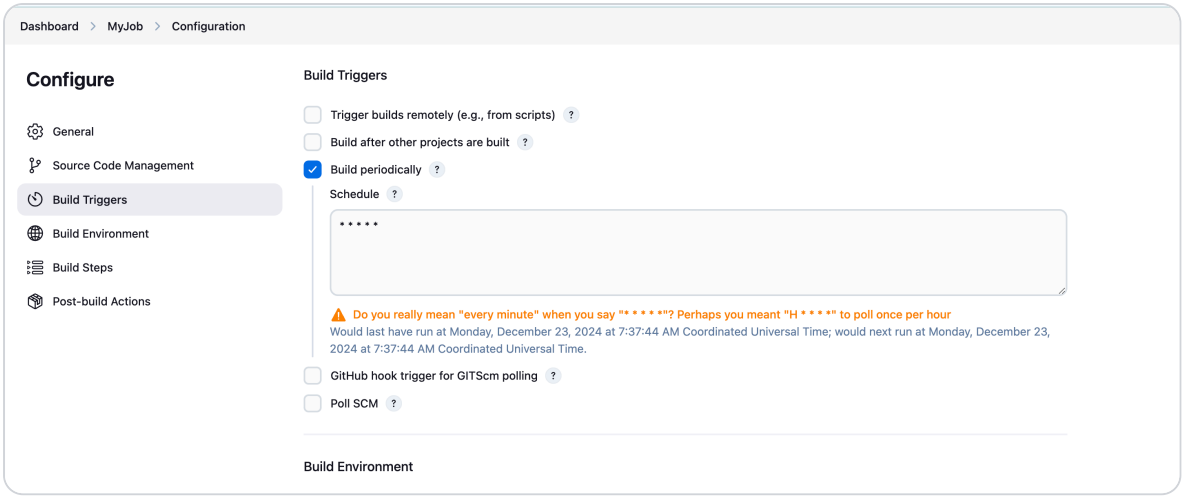
How to Integrate Build Periodically with Jenkins

Step 1: Configure Jenkins Job

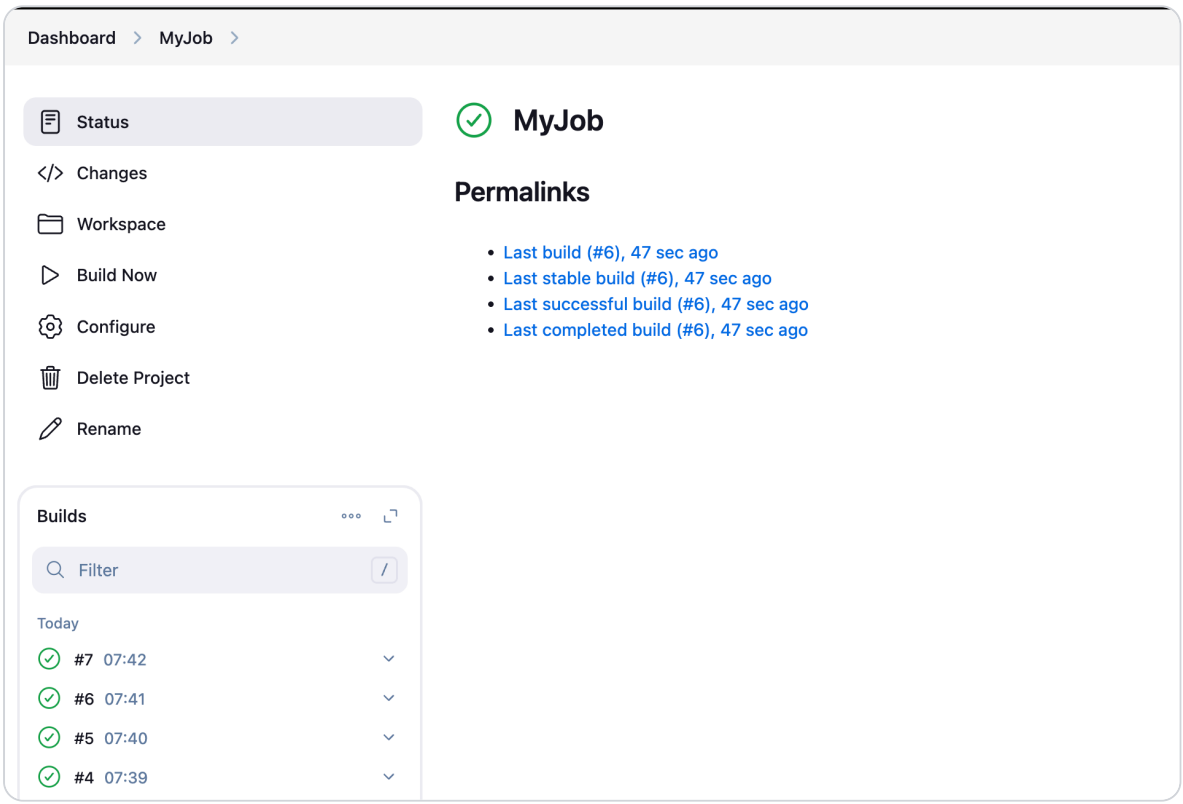
1. Go to the Jenkins dashboard and open the job configuration page.
2. Scroll to the **Build Triggers** section.
3. Select the checkbox for **Build periodically**.

Step 3: Define the Schedule

- In the **Schedule** text box, specify the frequency using cron-like syntax.



save the **job.**



Now Jenkins will trigger the jobs automatically, it is independent of code i.e.. even if we will not make any changes in code, it will trigger the jobs based on the schedule.

Difference b/w Webhooks, Build Periodically & Poll SCM

Scenario: Automating Builds for a GitHub Repository

1. **Webhooks**:
    - A developer pushes code to the `main` branch.
    - GitHub sends a webhook request to Jenkins, triggering an immediate build.
    - Ideal for active development teams.
2. **Poll SCM**:
    - Jenkins checks the repository every 5 minutes.
    - If changes are detected in the `main` branch, it triggers a build.
    - Slight delay but works even if webhooks are unavailable.
3. **Build Periodically**:
    - Jenkins triggers a build every night at midnight.
    - Useful for nightly regression testing or scheduled tasks unrelated to code changes.

Conclusion

Integrating Git with Jenkins is a game-changer for modern software development, enabling teams to streamline CI workflows. By following this guide, you can harness the power of automation to deliver high-quality software faster.

*If you enjoy stories that help you learn, live, and work better, **consider subscribing**. If this article provided you with value, please **support my work** — only if you can afford it. You can also **connect with me on LinkedIn**. Thank you!*